

Error Handling and Resilient Self-Healing Architecture

This document specifies the codebase-level programmatic error-handling, self-healing, string-normalization, and atomic database rollback layers implemented across the system pipelines.

Centralized Console Exception Briefing

To prevent verbose, intimidating tracebacks from cluttering the developer's command-line interface, all user-facing console failure logs are run through a centralized exception briefer. This utility maps complex system-level or API-level exceptions into clean, single-word uppercase indicators:

- **API Connection Failures:** Symmetrically maps to low-level pre-parsed codes extracted from transport dropouts and API timeouts (such as `"504 DEADLINE_EXCEEDED"`, `"TIMEOUT"`, `"CONNECTION_DROP"`, or `"NETWORK_READ_ERROR"`).
- **JSON Parsing Deviations:** Automatically maps standard library value error types to `"JSON_PARSE_ERROR"` or `"JSON_MISSING_ERROR"` based on exception string keywords.
- **Fallback Indicators:** Falls back to returning the uppercase representation of the exception's class name.

Centralized CLI Exception Handler

Symmetrically, we consolidate all unrecoverable CLI error catching and process termination inside a single unified utility. This utility: 1. **Extracts Briefs:** Automatically invokes the exception briefer to isolate the error's conceptually briefed uppercase indicator. 2. **Renders Milestones:** Output the standardized, unescaped red milestone to stderr. 3. **Prints Warning Suffixes:** Conditionally prints standard warning blocks (like index build abort alerts). 4. **Enforces Exit Terminology:** Maps preflight connection issues directly to the preflight exit status and other unhandled failures to the general failure exit status.

Visual Captioning Self-Healing JSON Parsing and Repair

During bulk, parallel region captioning, probabilistic model generations occasionally truncate or deviate from standard formatting under API gateway limits. To prevent expensive, high-fidelity caption losses, the interaction layer deploys two interlocking self-healing defense barriers:

Brace-Counting JSON Extractor

Surgically isolates the first valid block based on balanced brace weights, discarding all conversational chain-of-thought preambles and trailing junk. Symmetrically, it implements two stream-salvage algorithms: * **Omitted Brace Append:** If the stream terminates abruptly leaving exactly one top-level brace open, the extractor programmatically appends a closing brace to structurally seal the object. *

Unterminated String Salvage: If the count of non-escaped double quotes inside an open, unbalanced block is odd, the extractor mathematically proves the stream cut off mid-string literal. Symmetrically, it automatically appends a closing double quote first, strips any trailing commas, and then appends a closing brace to successfully salvage the truncated response.

Universal Quote and Control Character Repair

Prior to parsing, a regex-driven pipeline scans the raw string to escape unescaped internal double quotes inside standard JSON string fields. Symmetrically, it replaces raw, unescaped newlines and carriage returns inside string boundaries with standard escaped literals, completely neutralizing decoding errors. Symmetrically, this repair pipeline also recursively escapes all unescaped quotes, raw newlines, and carriage returns inside list elements of the `violations` array to prevent standard library decoder crashes under truncated model outputs.

Metadata Layer Assignment and Document Layer Proposal String Normalization and Validation

Probabilistic model predictions occasionally return labels containing unescaped quotes, stray punctuation, backslashes, or whitespace separators. Symmetrically, we guard both the document-level and asset-level classification steps with a shared, dual-repair validation layer:

Robust String Normalization

Preemptively cleanses model output strings by stripping enclosing quotes, replacing stray backslashes, splitting on newlines, and isolating the primary word token before matching against the layer ontology names list. Symmetrically, this is implemented via a centralized shared utility that cleanses and normalizes the prediction.

Result: Automatically self-heals corrupted outputs directly across both classification steps, preventing classification skips!

Thread-Safe Validation Error Tracking and Flag Retention

If a normalized layer string remains invalid after cleansing, the engine does not silently return or clear verification states. Symmetrically, the engine:

1. **Retains Verification Flags:** Retains the pending state for any document or asset that fails, ensuring the operator can safely re-run the classification command to retry failed items.
2. **Tracks Error Summaries:** Intercepts the error and thread-safely increments the summary error counter.
3. **Prints Warning Indicators:** Displays an icon-free solid red warning detailing the specific failure.
4. **Triggers CLI Completion Warning:** Correctly triggers our standards-compliant orange completed-warning upon completion, notifying the operator of the incomplete classifications and blocking downstream processing.

Symmetrical Exception-Handling Boundaries

To enforce maximum robustness and state integrity, the system maps the handling of unrecoverable errors into exactly three distinct execution classes based on the presence of thread-separation:

Symmetrical Behavior for Agentic Loops (Socket-Separated Concurrency)

This includes the macro-discovery loop, the micro-subdivision loop, and the decoupled verification loop. For detailed systems-level thread-local socket cache isolation mechanics, see our concurrency specification document: * **The Concurrency Context:** These are multi-turn, conversational, agentic loops operating concurrently inside a parallel thread pool. * **The Behavior:** Symmetrically, we enforce strict exception differentiation inside our worker threads:

1. *Expected Exceptions (Connection/Timeout/Turns/JSON Parsing):* Caught locally inside the worker thread. Because a thread-local socket pool cannot cleanly close sibling connections, we

renounce forcefully crashing the entire pool. The thread gracefully catches the expected exception, writes its local progress logs, rolls back its specific on-disk page census checklist item to `PENDING` via rollback utilities, preserves drafted regions on disk, and returns cleanly so sibling threads can finish executing safely. 2.

Unclassified Exceptions (Developer Bugs): Symmetrically bypasses all thread-local filters! Symmetrically, standard Python bugs are allowed to bubble out of the worker thread unimpeded, immediately crashing the thread pool, halting execution, and exiting with our red failure milestone, exposing the real traceback to the user!

Symmetrical Behavior for Single-Thread Commands

This includes the document ingestion step, the deep-integrity verification step, the final package exporting step, the single-query step, and the interactive conversational REPL sessions:

* **The Concurrency Context:** These are single-threaded, sequential local-first operations executing strictly on the main thread (exactly 1 thread). * **The Behavior:** Symmetrically, the process immediately fails-fast:

1. *Immediate Process Halt:* Because there is absolute zero thread-separation, trying to isolate the exception locally is useless. The entire command execution halts instantly upon encountering an unrecoverable exception or developer bug. 2.

Resource Closure Guarantees: Symmetrically, to prevent resource or read leaks, local-first file operations (such as FTS5 search queries) are wrapped in try-finally structures to guarantee that database connections are closed on any exception paths. 3. *Unclassified Red Failure Milestone:* The command-line exception handler immediately intercepts the error, funnels the raw exception through our exception classification layer, and outputs a standardized, unescaped red failure milestone to stderr. 4. *General Exit Termination:* The process terminates cleanly with a process-level general error code, ensuring no broken state or misleading terminal displays are generated.

Symmetrical Behavior for Standard Parallel Steps

This includes the document layer classification step, the asset layer assignment step, and the asset captioning step:

* **The Concurrency Context:** These are concurrent bulk-operations executing inside a thread pool, but do not represent independent page-agent conversational loops. * **The Behavior:** Symmetrically, we differentiate unclassified developer errors from server overloads inside our parallel execution engines: 1. *Unclassified Fail-Fast (Developer Bugs):* Symmetrically, during standard parallel steps, the executor utilizes a shared thread-safe shutdown event flag. The first thread to hit an unclassified developer error sets this event flag, immediately aborts further scheduling of queued tasks, re-raises the exception to crash the pool, and exits with a standardized red failure milestone at the command-line interface. 2.

Connection Resilience (Timeout/Quota): Symmetrically, during standard parallel steps, any transient timeout or quota limit exception is caught locally inside the worker thread. Symmetrically, the thread halts only its own task execution, thread-safely increments the summary error count, prints a standardized orange completed-warning, and allows all other parallel sibling threads to continue executing and serialize successfully.

Symmetrical Behavior for Hybrid Decoupled Indexing

This includes the index building step (prose text chunks, visual captions, and image crops indexing):

- * **The Concurrency Context**: These are decoupled bulk-operations separating asynchronous network translation threads (MIMD) from C++ model matrix vectorizations (SIMD).
- * **The Behavior**: Symmetrically, we differentiate unclassified compiler and vectorizer errors from remote API translation drops:

1. *Unclassified Fail-Fast (Developer/Model Bugs)*: Symmetrically, if any unclassified compiler, shape mismatch, or vectorizer exception occurs during the local matrix embedding passes (BGE or SigLIP batch encodings), the execution immediately aborts and re-raises the exception, exiting with a standardized red failure milestone.
2. *Connection Resilience (Translation Timeouts)*: Symmetrically, transient Category B network translation failures (e.g. Gemini Gateway Timeouts) are caught locally in the translation pool. Symmetrically, the affected document is skipped and rolled back from partial index writes, thread-safely logging an orange completed-warning while letting sibling translation threads complete successfully.
3. *Incremental Cache Preservations*: Symmetrically, we preserve the SQLite FTS5 database and ChromaDB vector store directories, committing only fully completed documents. Symmetrically, this allows subsequent indexing runs to resume cleanly from the last successfully indexed document state.